

Machine Learning

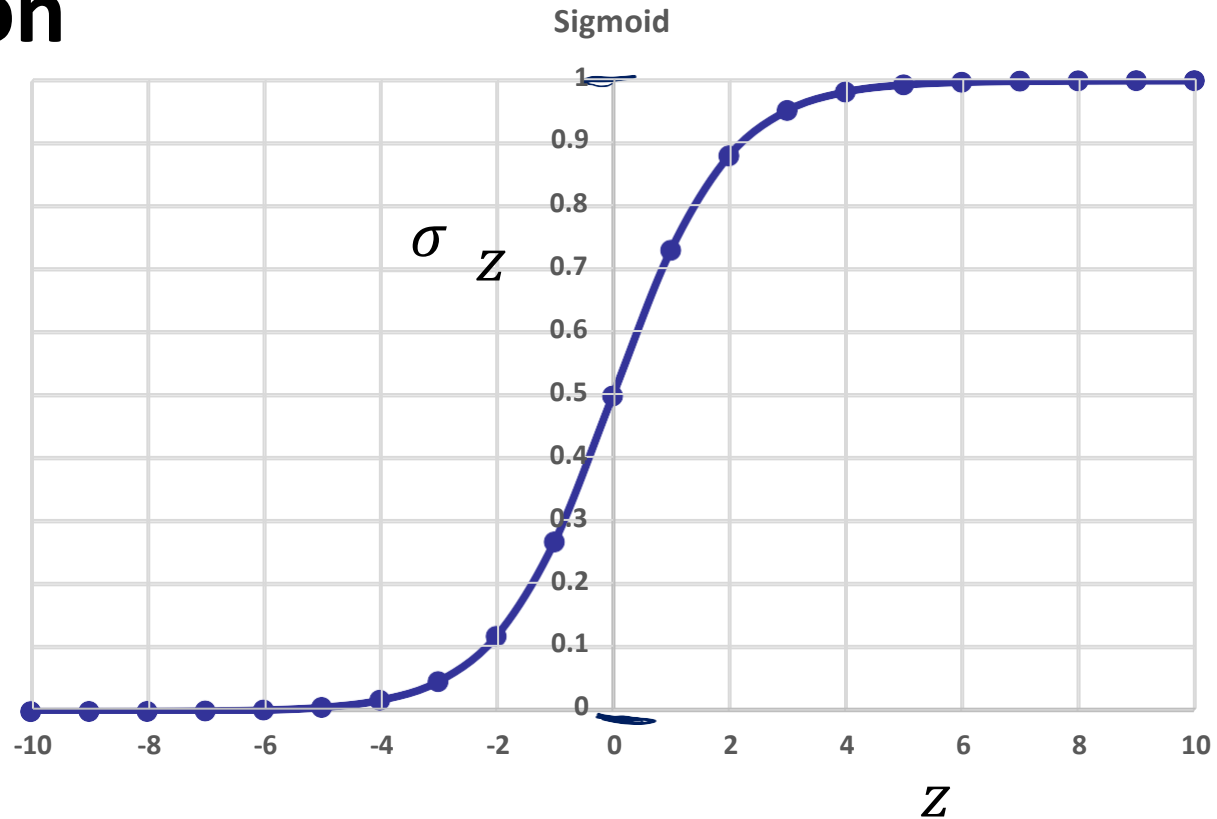
Instructor: Hafiz Abdul Rehman

Lecture 15: Logistic Regression – Decision Boundary

Logistic regression

$$z = \theta^T X$$

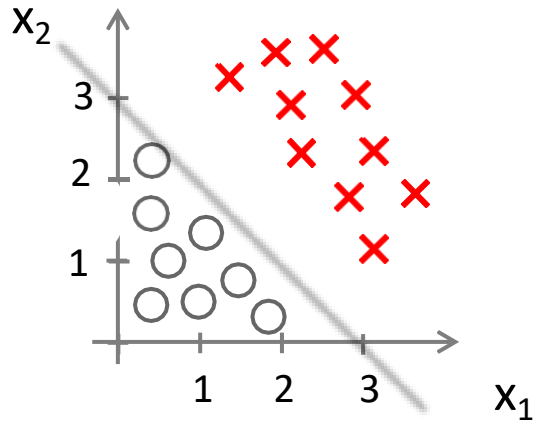
$$h(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Suppose, predict “y=1” if $\sigma(z) \geq 0.5$
i.e. $\theta^T X \geq 0$

predict “y=0” if $\sigma(z) < 0.5$
i.e. $\theta^T X < 0$





$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

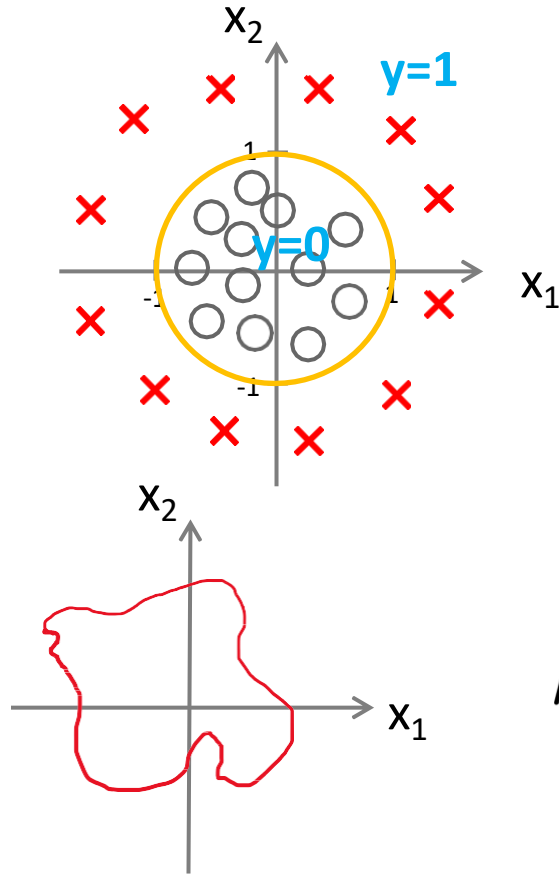
Therefore, predict $y=1$, if $-3 + x_1 + x_2 \geq 0$
 $x_1 + x_2 \geq 3$

Where the decision boundary represents:

$$x_1 + x_2 = 3$$



Non-linear decision boundaries



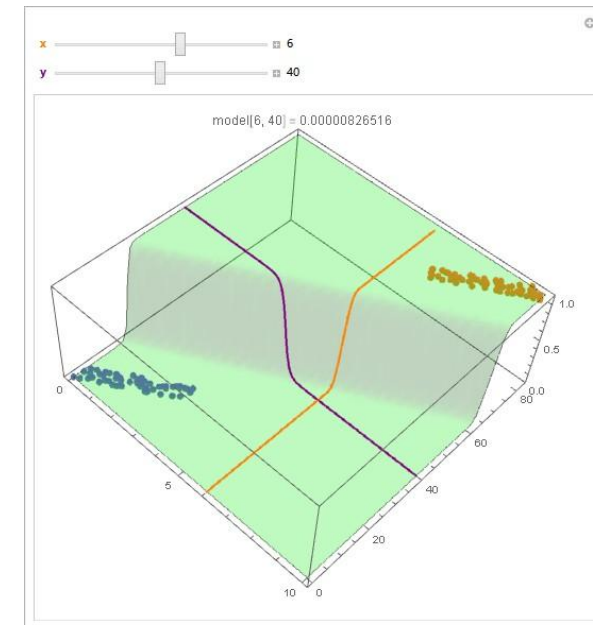
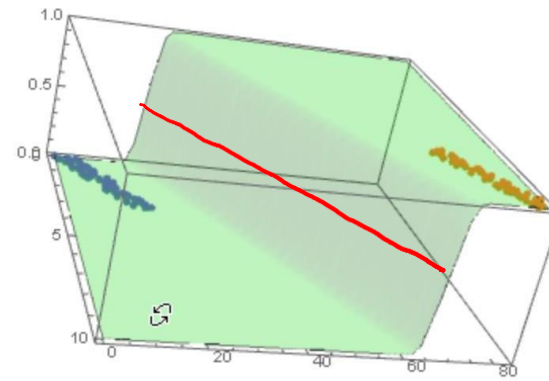
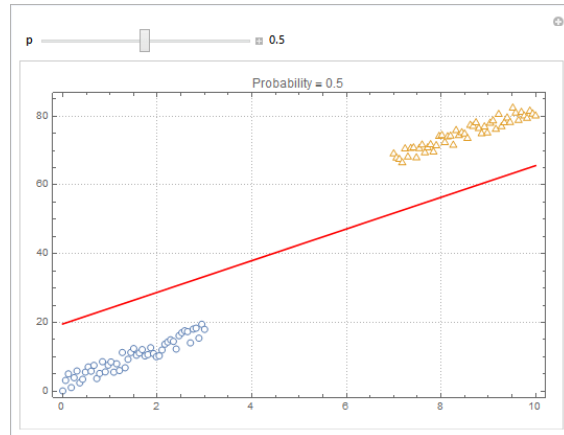
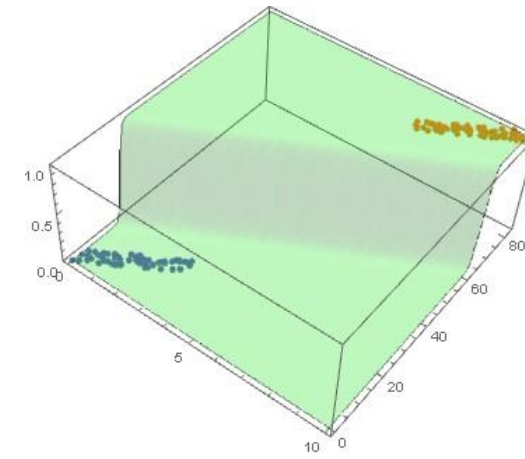
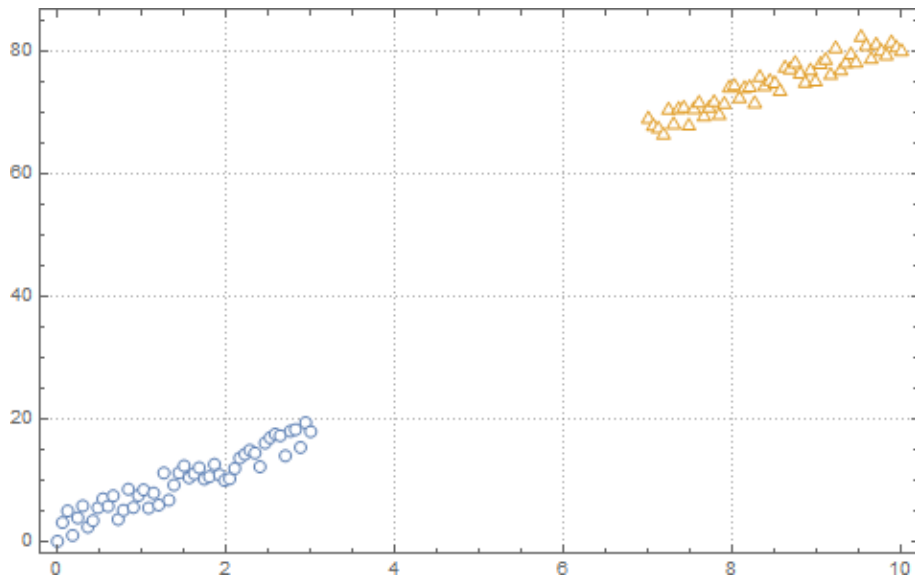
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict $y = 1$ if $-1 + x_1^2 + x_2^2 \geq 0$
 $x_1^2 + x_2^2 \geq 1$

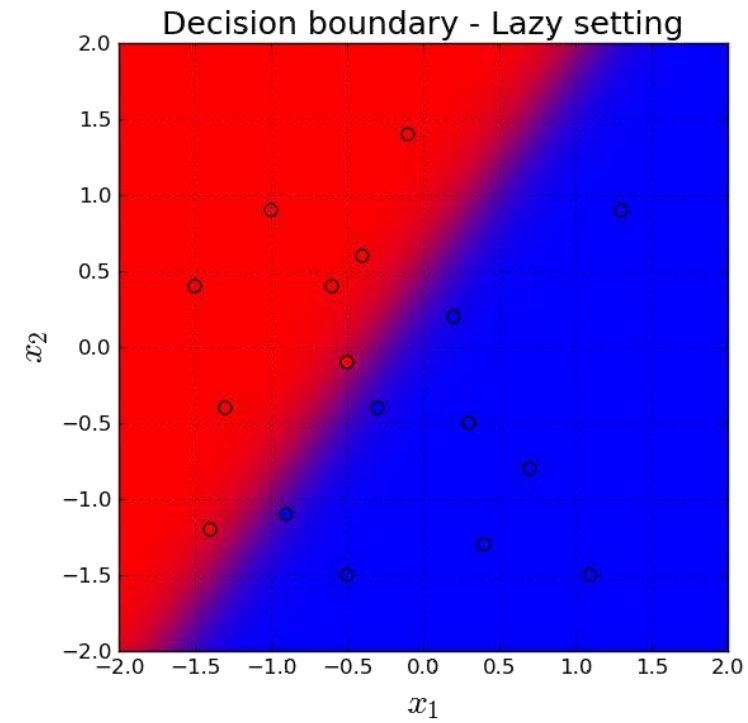
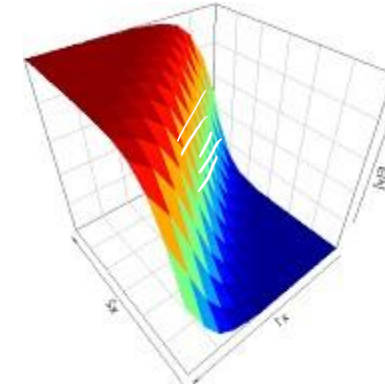
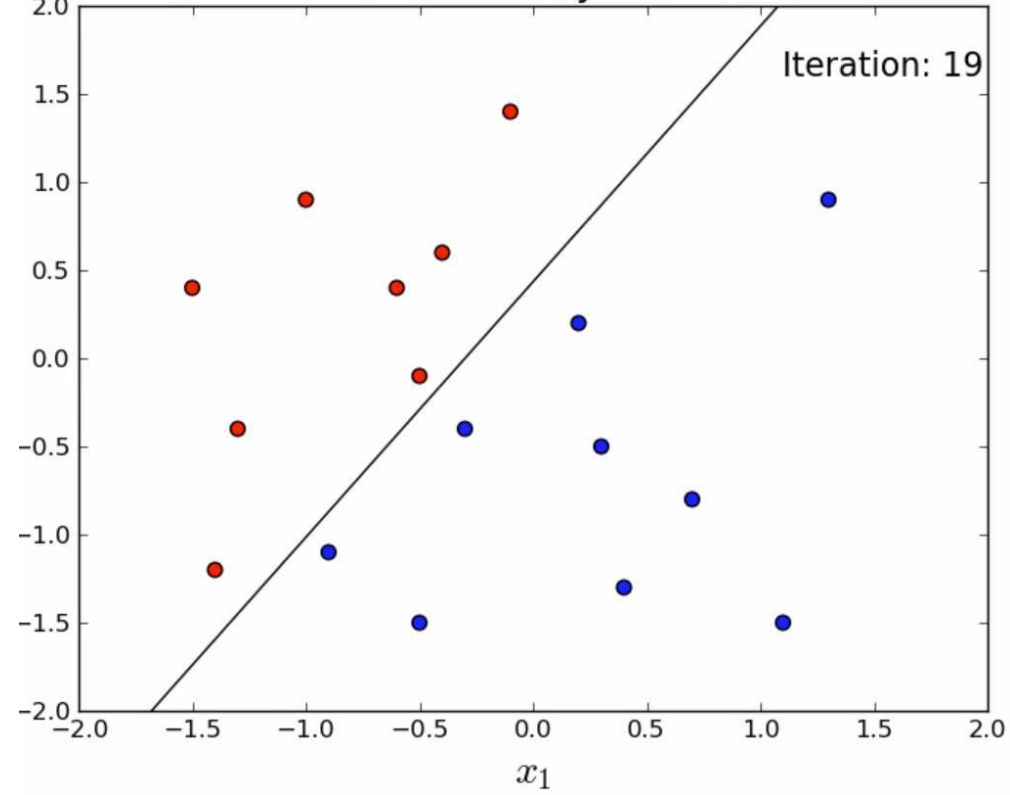
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

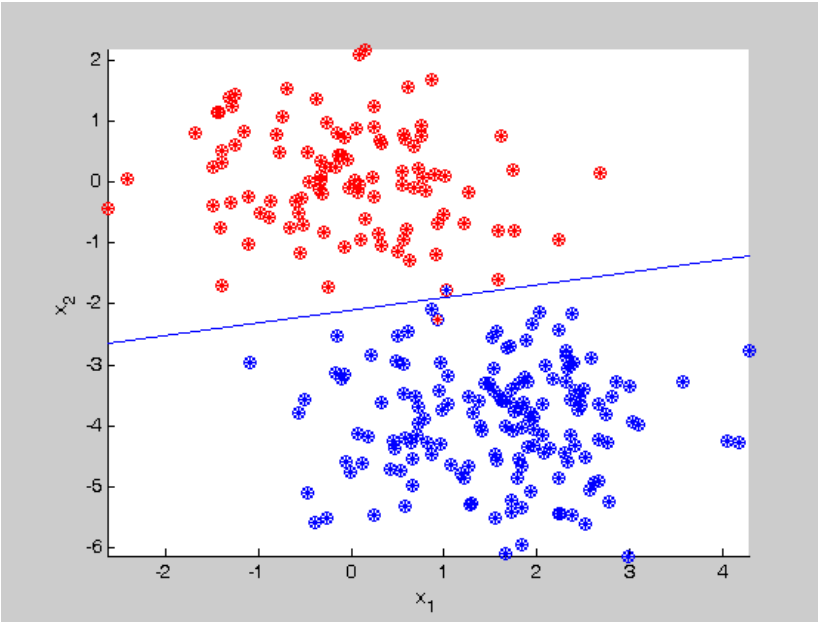




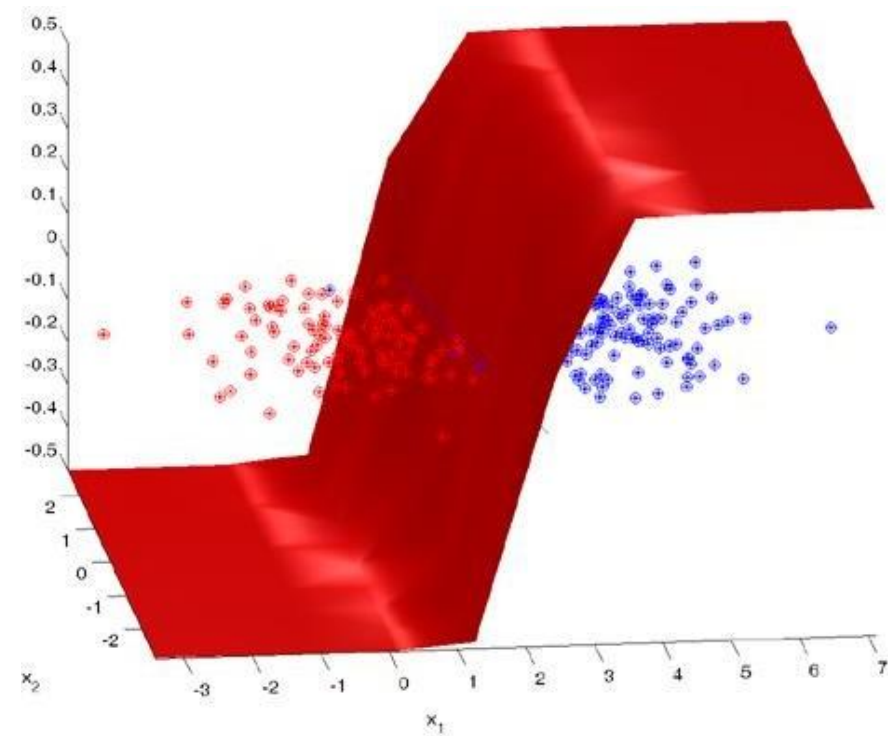
<https://datascience.stackexchange.com/questions/49573/how-to-plot-logistic-regression-decision-boundary>

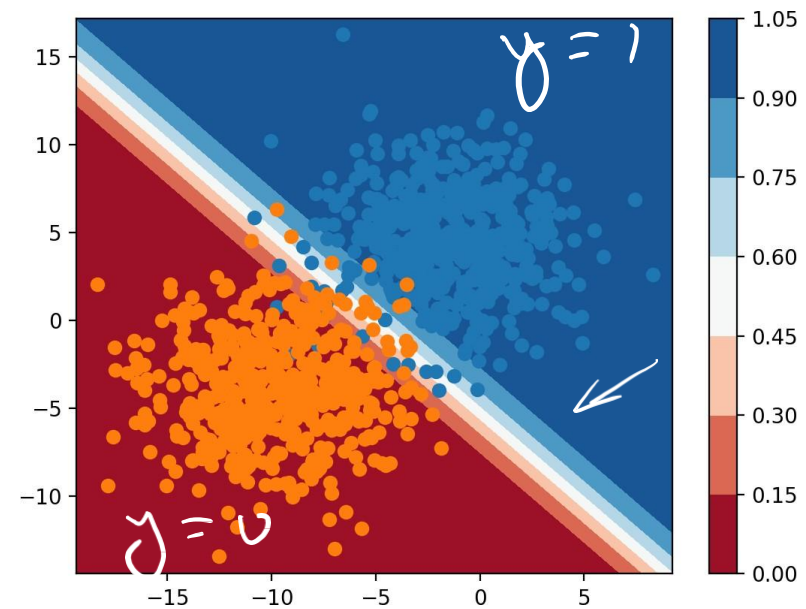
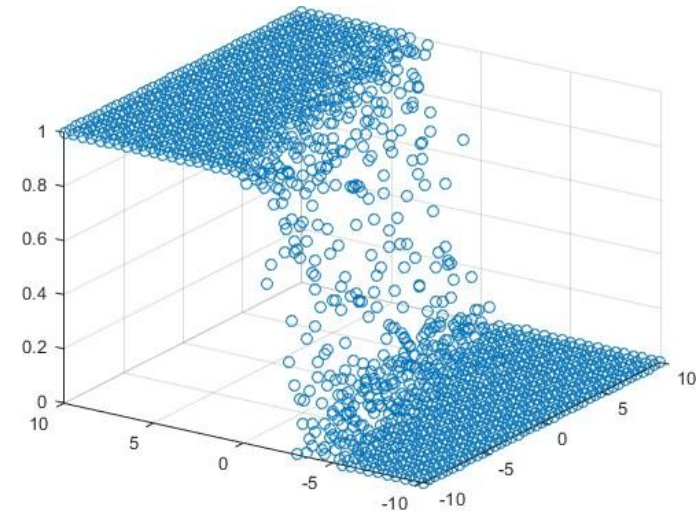
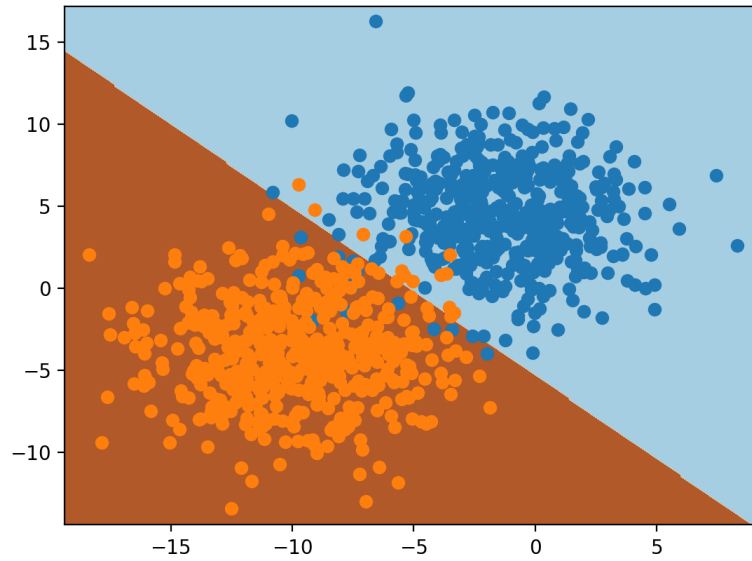






<http://strijev.com/sources/demoDataGen.php>





<https://machinelearningmastery.com/plot-a-decision-surface-for-machine-learning/>
<https://stackoverflow.com/questions/29360872/fitting-3d-sigmoid-to-data>



Cost Function



Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

m examples

$$x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

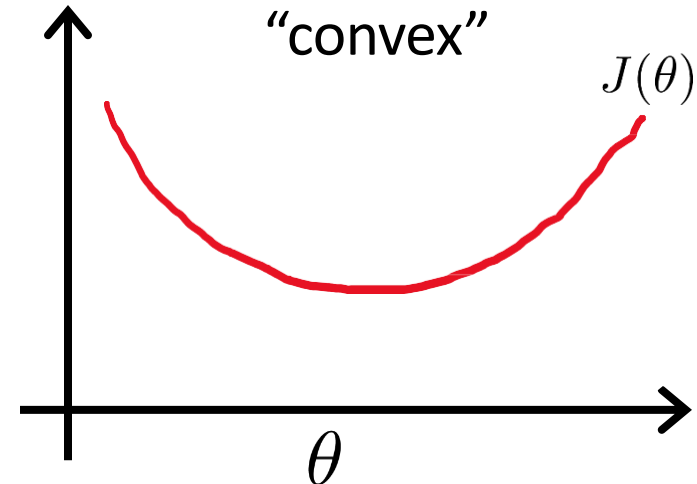
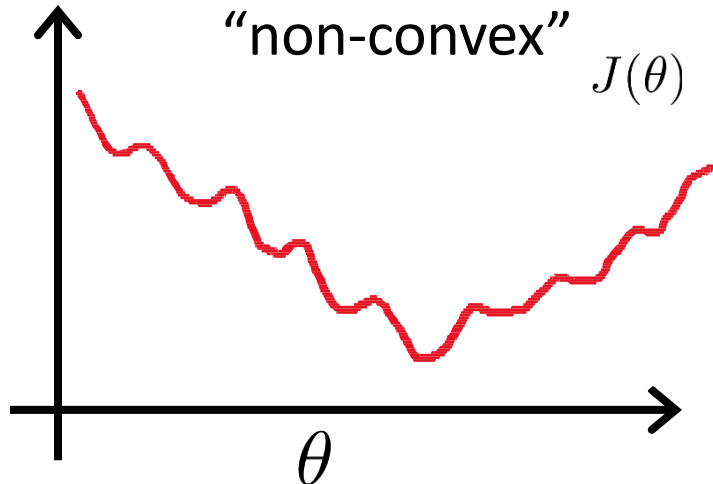


Cost function

$$\text{cost}(h_{\theta}(x), y)$$

Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \boxed{\frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2}$

$$\text{cost}(h_{\theta}(x), y) = \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2} \left(\frac{1}{1 + e^{-\theta^T x^{(i)}}} - y^{(i)} \right)^2$$



Our goals for cost function

- Differentiable
- Convex

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x^{(i)}}} \text{ varies between 0 and 1.}$$

- Punish incorrect answers with high cost
 - $y=1$, $h_{\theta}(x) \rightarrow 0$
 - $y=0$, $h_{\theta}(x) \rightarrow 1$
- Reward correct answers with a low cost
 - $y=1$, $h_{\theta}(x) \rightarrow 1$
 - $y=0$, $h_{\theta}(x) \rightarrow 0$

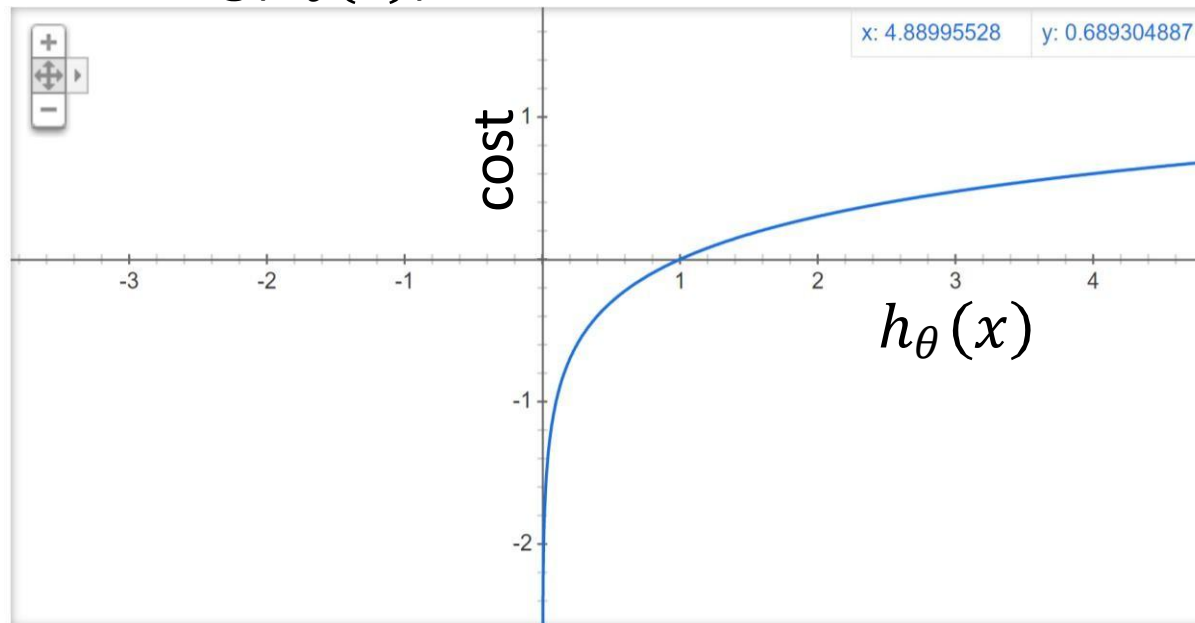


Plot it live: <https://www.desmos.com/calculator>

- Rewards $h_{\theta}(x) = 1$, with cost = 0
- Rewards $h_{\theta}(x) \rightarrow 0$ even further with asymptotically smaller costs

y=1

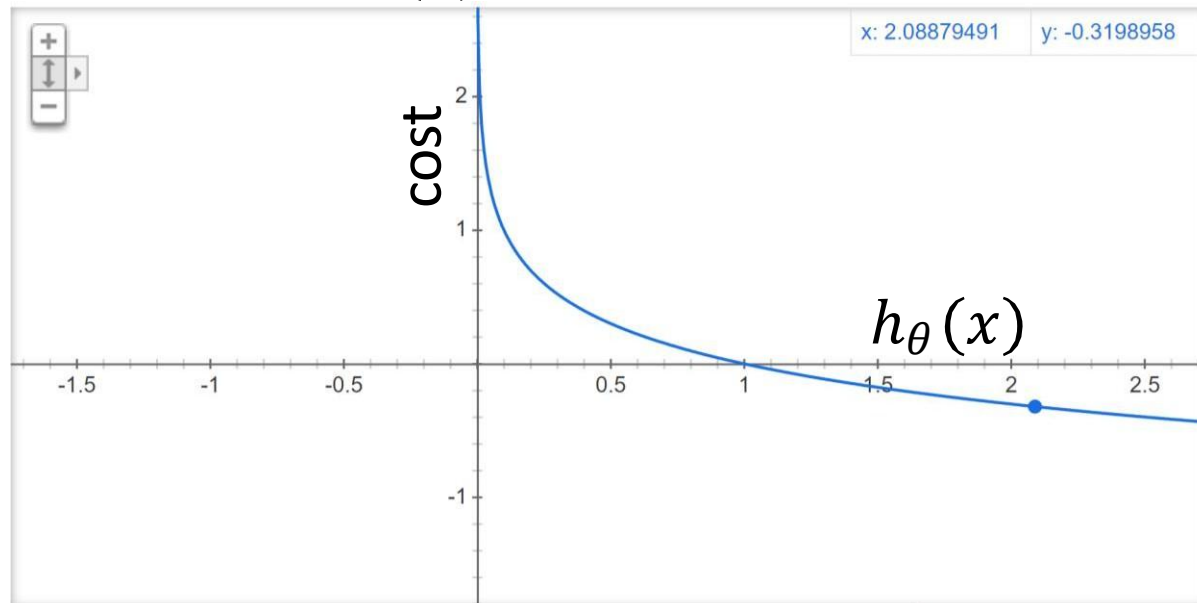
$$\text{cost} = \log(h_{\theta}(x))$$



- Rewards $h_{\theta}(x) = 1$, with cost = 0
- Punishes $h_{\theta}(x) \rightarrow 0$ with asymptotically higher costs

$y=1$

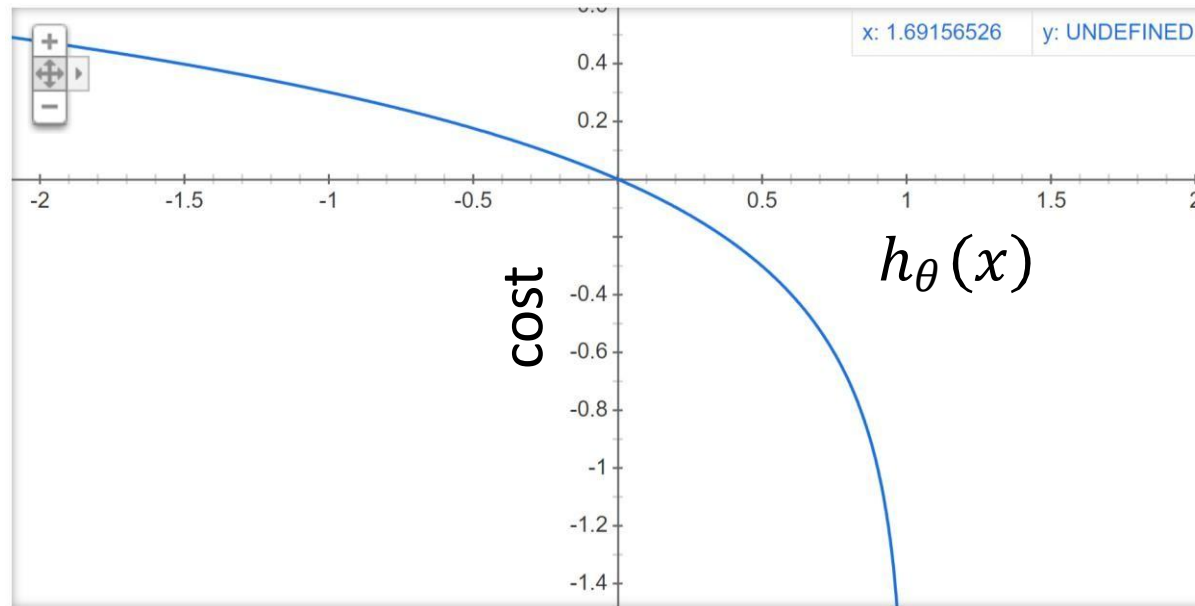
$$\text{cost} = -\log(h_{\theta}(x))$$



- Rewards $h_{\theta}(x) = 0$, with cost = 0
- Rewards $h_{\theta}(x) \rightarrow 1$ even further with asymptotically smaller costs

$y=0$

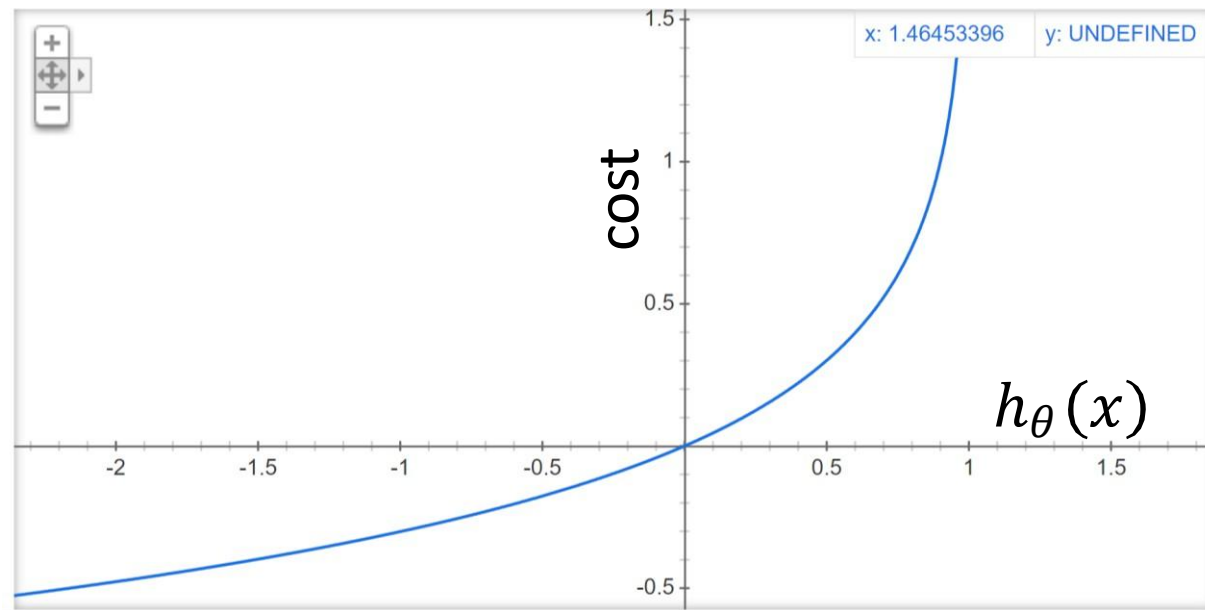
$$\text{cost} = \log(1 - h_{\theta}(x))$$



- Rewards $h_{\theta}(x) = 0$, with cost = 0
- Rewards $h_{\theta}(x) \rightarrow 1$ even further with asymptotically smaller costs

$y=0$

$$\text{cost} = -\log(1 - h_{\theta}(x))$$



Logistic Regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Can we combine these and create a switch that turns on the correct cost values for the values of y ?



Logistic Regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Can we combine these and create a switch that turns on the correct cost values for the values of y ?

Note: $y = 0$ or 1 always (Bernoulli Distribution)

$$\text{cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

$$\text{if } y = 1: \text{cost}(h_{\theta}(x), y) = -\log(h_{\theta}(x))$$

$$\text{if } y = 0: \text{cost}(h_{\theta}(x), y) = -\log(1 - h_{\theta}(x))$$



Logistic Regression cost function

$$\text{cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

- We have created a loss function that prefers the correct class labels of the **training examples** to be more likely
 - It **estimates** the model parameters to **maximize** the **likelihood** of the training set
 - This is called **maximum likelihood estimation** (MLE)
- We choose the parameters θ_i that maximize the log probability of the true y labels in the training data given the observations x .
- The resulting loss function is the negative log likelihood loss, generally called the **cross-entropy loss**.



Logistic Regression cost function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

To fit parameters θ :

$$\min_{\theta} J(\theta)$$

To make a prediction given new x :

$$\text{Output } h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} \quad \text{gives } p(y = 1|x; \theta)$$



Gradient Descent



Batch Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

} (simultaneously update all θ_j)

$$\frac{\partial}{\partial \theta_j} (J(\theta)) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(see SLP3 for derivation)



Batch Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

Algorithm looks identical to linear regression!



Batch Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {
 $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$
 (simultaneously update all θ_j)
}



Stochastic Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {

Repeat for each training instance $(x^{(i)}, y^{(i)})$, in random order{

$$\theta_j := \theta_j - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update all θ_j)

}

}



Mini-batch Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {

Repeat for each randomly selected minibatch of size k {

$$\theta_j := \theta_j - \alpha \frac{1}{k} \sum_{i=1}^k (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

}

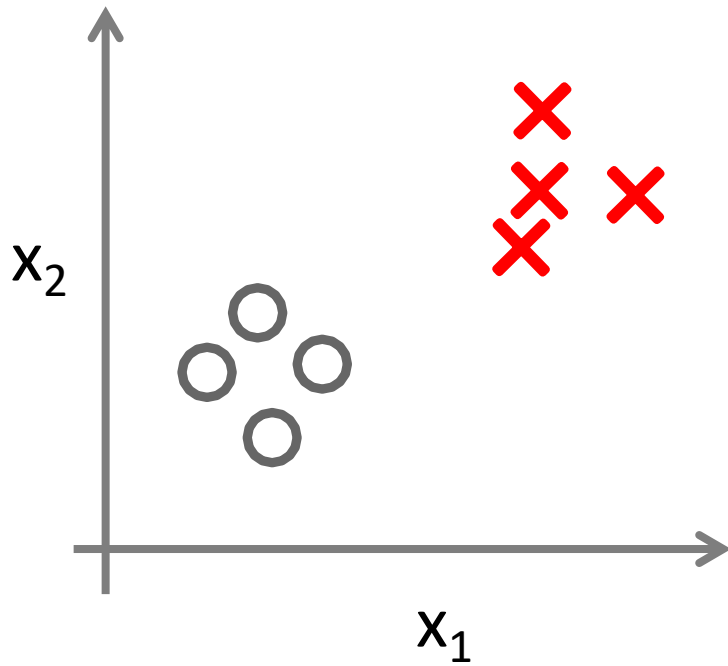
}



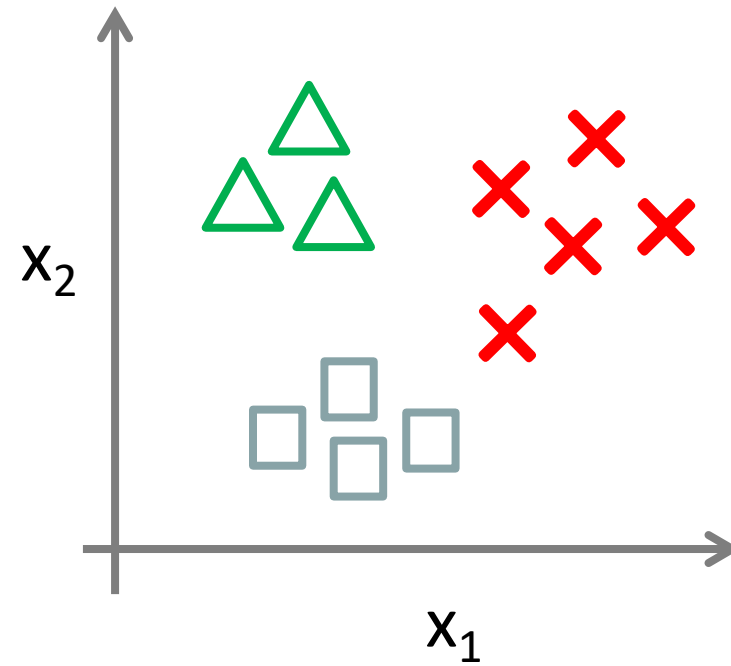
Multiclass Classification



Binary classification:



Multi-class classification:



Multiclass classification

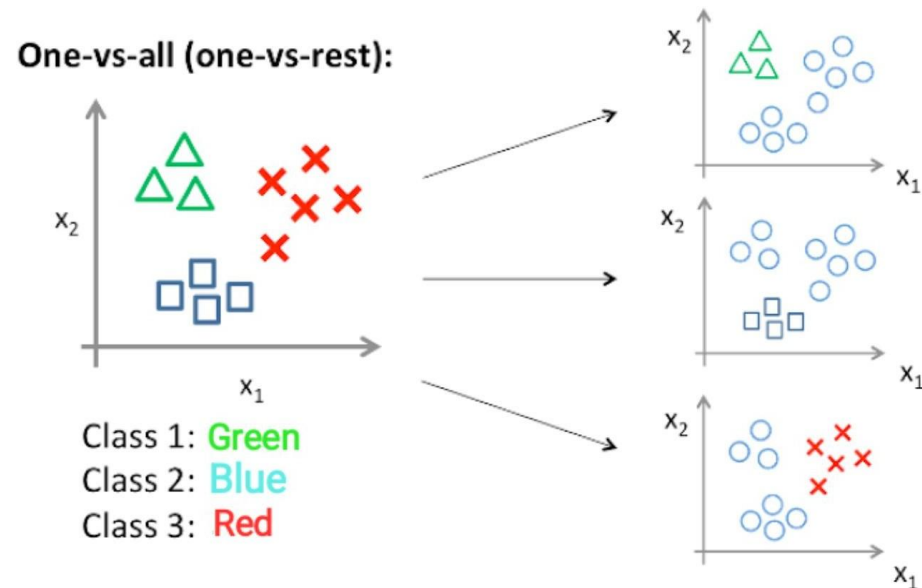
- Logistic Regression inherently models binary classification and needs meta-strategies for multiclass classification
- Multiclass classification
 - Multiple class labels are present in the dataset.
- Binary classifiers for multiclass classification:
 - One vs Rest (aka One vs All)
 - Splits a multi-class classification into one binary classification problem per class.
 - n -class instances then n binary classifier models
 - One vs One
 - Splits a multi-class classification into one binary classification problem per each pair of classes.
 - n -class instances then $\binom{n}{2} = \frac{n * (n-1)}{2}$ binary classifier models



One vs All (One vs Rest)

- One classifier per class ($x \in class_i$ vs $x \notin class_i$) for all n classes
- In the scoring phase all the n classifiers predict the probability of each class and class with highest probability is selected

- Classifier 1:- [Green] vs [Red, Blue]
- Classifier 2:- [Blue] vs [Green, Red]
- Classifier 3:- [Red] vs [Blue, Green]



Create separate data sets for training

Main Dataset

Features			Classes
x1	x2	x3	G
x4	x5	x6	B
x7	x8	x9	R
x10	x11	x12	G
x13	x14	x15	B
x16	x17	x18	R

Training Dataset 1
Class :- Green

Features			Green
x1	x2	x3	+1
x4	x5	x6	-1
x7	x8	x9	-1
x10	x11	x12	+1
x13	x14	x15	-1
x16	x17	x18	-1

Class 1 :- Green Class 2 :- Blue Class 3 :- Red

Training Dataset 2
Class :- Blue

Features			Blue
x1	x2	x3	-1
x4	x5	x6	+1
x7	x8	x9	-1
x10	x11	x12	-1
x13	x14	x15	+1
x16	x17	x18	-1

Training Dataset 3
Class :- Red

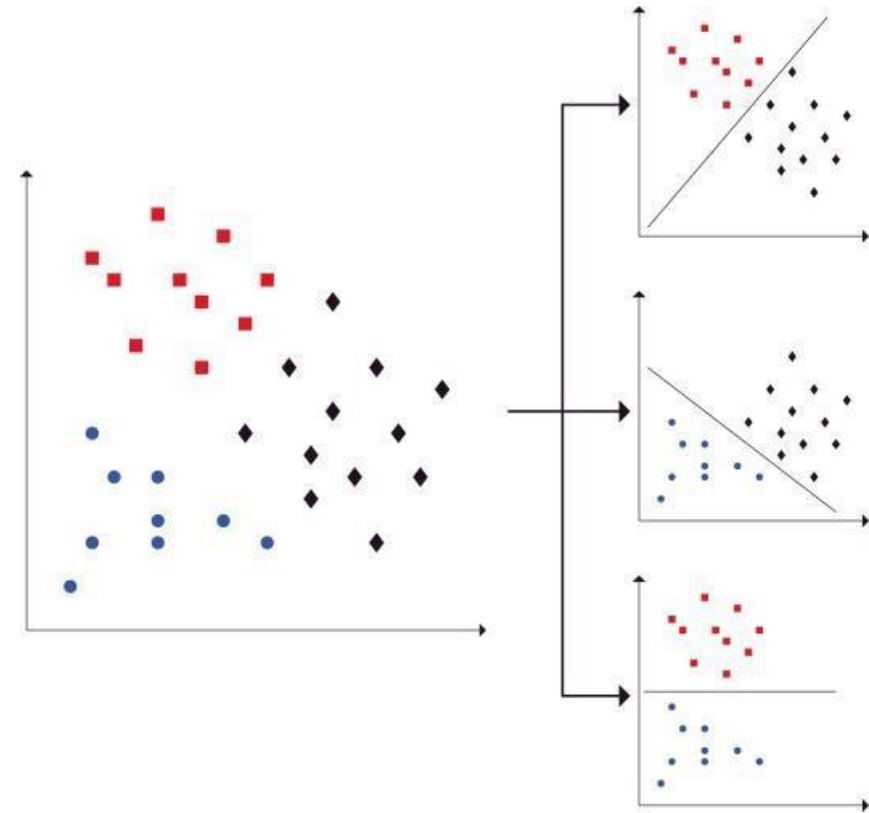
Features			Red
x1	x2	x3	-1
x4	x5	x6	-1
x7	x8	x9	+1
x10	x11	x12	-1
x13	x14	x15	-1
x16	x17	x18	+1

<https://towardsdatascience.com/multi-class-classification-one-vs-all-one-vs-one-94daed32a87b>



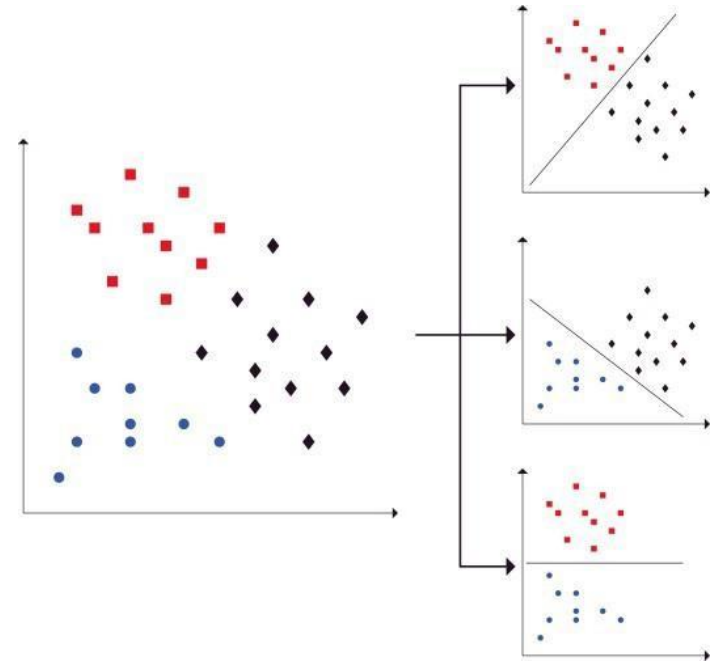
One vs One

- For n classes, we must generate $\frac{N * (N-1)}{2}$ binary classifier models
- Using this classification approach, we split the primary dataset into one dataset for each class vs every other class.
- Classifier 1: Black vs. Blue
- Classifier 2: Black vs. Red
- Classifier 3: Blue vs. Red



One vs One

- Each binary classifier predicts one class label and the model with the most predictions or votes is predicted by the one-vs-one strategy.
- If the binary classification models predict a numerical class membership, such as a probability, then the *argmax* of the sum of the scores (class with the largest sum score) is predicted as the class label.



oVr vs oVo

- oVr trains fewer classifiers and is faster overall
- oVo is less prone to imbalance in datasets

Thank you!

